

OPEN STUDIO: 28-DAY AGENTIC EXECUTION BLUEPRINT

Machine-Actionable Daily Engineering Plan for Antigravity One-Shot Execution

Target Stack: Tauri v2 (Rust) + React 18 (TS) + Monaco	Hardware Baseline: i7-1355U • 16GB RAM • RTX 3050 (4GB) & Apple M-Series
Inference: Ollama (SSE) + llama.cpp + MLX	Latency Targets: < 40ms Tab Ghost-Text • < 2s Frugal In-line Diffs
Security: 100% Air-Gapped Zero-Telemetry CI Verified	Cadence: 4 Weeks • 28 Daily Atomic Milestones for Agent Prompting

+-----+			
Open Studio Desktop Shell: Tauri v2 (Rust Backend + React UI) ~15MB binary, ~35MB RAM footprint			
+-----+			
Editor Core	File System Tree	Embedded Terminal	Diff Review
Monaco Editor	Notify Watcher + SQLite	portable-pty + xterm.js	Multi-File Hunk
+-----+			
Language Intelligence	Rust LSP Client (JSON-RPC) -> rust-analyzer, pyright, tsserver, gopls		
+-----+			
Codebase Context Index	Tree-sitter AST Slicer (800 tok) + nomic-embed-text + SQLite-vec + BM25		
+-----+			
Dynamic Agent Core	Dynamic Model Router (keep_alive VRAM manager) + Shadow Git Checkpoints		
+-----+			
Local Inference Gateway	Ollama API (SSE) + llama.cpp server + MLX Metal (Apple Silicon)		
+-----+			

WEEK 1: FOUNDATIONAL CORE & EDITOR INFRASTRUCTURE

DAY 1: PROJECT SCAFFOLDING & TAURI V2 SHELL

Objective:	Setup production Tauri v2 workspace with Rust core, Vite + React + TS, Tailwind CSS in air-gapped dark aesthetic.
Deliverables:	src-tauri/Cargo.toml (Tauri 2.x, tokio, serde), tauri.conf.json (offline CSP), AppSystemInfo IPC bridge.
Interface / Contract:	get_system_info() -> Result returning OS, arch, total RAM.
Antigravity Prompt:	"Scaffold Tauri v2 + React 18 + TS + Tailwind. Implement get_system_info IPC command. Configure strict offline CSP. Verify cargo check & npm run build."
Test Command:	cd src-tauri && cargo check && npm run build

DAY 2: MONACO EDITOR CORE & LAYOUT ENGINE

Objective:	Integrate Monaco Editor with multi-tab switching, split views (H/V), dirty tracking, and local worker bundling.
Deliverables:	MonacoEditor.tsx (open-studio-dark theme), editorStore.ts (Zustand), TabBar.tsx, split pane CSS container.
Interface / Contract:	interface EditorBuffer { id: string; filePath: string; content: string; isDirty: boolean; }
Antigravity Prompt:	"Bundle Monaco locally without CDNs. Implement Zustand editorStore for open buffers, tabs, and split panes. Wire Ctrl+S, Ctrl+W, Ctrl+\.."
Test Command:	npm run test -- -t 'editorStore' && npm run build

DAY 3: NATIVE FILE EXPLORER & VIRTUAL WORKSPACE TREE

Objective:	Build native file watcher and recursive folder tree capable of 50,000+ files without UI lag.
Deliverables:	src-tauri/src/fs/ (notify-debouncer-mini, walkdir), FileTree.tsx (virtualized list, git status badges).
Interface / Contract:	<code>read_workspace_tree(root: String, depth: usize) -> Result; emits 'workspace-fs-change'.</code>
Antigravity Prompt:	<i>"Implement read_workspace_tree respecting .gitignore. Stream fs-change events via notify. Render virtualized tree linking to editorStore."</i>
Test Command:	<code>cd src-tauri && cargo test fs_tree && npm run test -- -t 'FileTree'</code>

DAY 4: EMBEDDED TERMINAL INFRASTRUCTURE (RUST PTY + XTERM.JS)

Objective:	Embed native cross-platform PTY terminal supporting PowerShell/cmd and bash/zsh with ANSI color streams.
Deliverables:	src-tauri/src/terminal/ (portable-pty), TerminalPanel.tsx (xterm.js + fit addon).
Interface / Contract:	<code>spawn_terminal_session(id: String, shell: Option), write_terminal_input(id, data), resize_terminal(id, cols, rows).</code>
Antigravity Prompt:	<i>"Build PTY backend with portable-pty crate. Stream terminal bytes to frontend xterm.js. Wire resize handler and input dispatch."</i>
Test Command:	<code>cd src-tauri && cargo test terminal_pty && npm run build</code>

DAY 5: LOCAL INFERENCE GATEWAY & STREAMING SSE CLIENT

Objective:	Build multi-backend async HTTP streaming client for Ollama, llama.cpp, and OpenAI endpoints with abort control.
Deliverables:	src-tauri/src/inference/ (request SSE client), stream_completion command emitting 'llm-chunk', abort_completion.
Interface / Contract:	<code>stream_completion(window, request_id, req: CompletionRequest) -> Result<(), String> with keep_alive support.</code>
Antigravity Prompt:	<i>"Implement Ollama streaming SSE gateway using request. Emit chunk events keyed by request_id. Implement cancellation via oneshot channels."</i>
Test Command:	<code>cd src-tauri && cargo test inference_gateway</code>

DAY 6: INLINE GHOST-TEXT AUTOCOMPLETE ENGINE

Objective:	Build sub-40ms inline Tab autocompletion inside Monaco powered by resident Qwen2.5-Coder-1.5B.
Deliverables:	inlineProvider.ts (Monaco InlineCompletionsProvider), FIM prompt formatter, 30ms keystroke debouncer.
Interface / Contract:	<code>formatFIMPrompt(prefix, suffix) -> '< fim_prefix >...< fim_suffix >...< fim_middle >'; Tab accepts.</code>
Antigravity Prompt:	<i>"Implement Monaco InlineCompletionsProvider. Extract cursor context, format Qwen FIM prompt, debounce 30ms, query Ollama (keep_alive: -1)."</i>
Test Command:	<code>npm run test -- -t 'inlineProvider'</code>

DAY 7: WEEK 1 INTEGRATION & LATENCY BENCHMARKS

Objective:	Unify editor, tree, terminal, and autocomplete; verify < 40ms TTFT and < 50MB shell RAM usage.
Deliverables:	src/App.tsx cohesive layout, bench/ttft.py benchmark harness, milestone tag v0.1.0-week1.
Interface / Contract:	<code>bench/ttft.py asserts average TTFT < 40ms across 50 requests; memory profiler verifies < 50MB idle.</code>
Antigravity Prompt:	<i>"Perform Week 1 integration pass. Connect FileTree, Monaco, Terminal, and Autocomplete. Run bench/ttft.py and verify sub-40ms latency."</i>
Test Command:	<code>npm run build && cd src-tauri && cargo check --release && python bench/ttft.py</code>

WEEK 2: TOKEN OPTIMIZER & IN-LINE DIFF ENGINE

DAY 8: FRUGAL SEARCH/REPLACE DIFF PROTOCOL & PARSER

Objective:	Build deterministic diff parser saving 98% output tokens by generating only changed lines.
Deliverables:	src-tauri/src/diff/frugal_parser.rs (regex + state machine parser), multi-hunk validator, apply_frugal_diff command.
Interface / Contract:	<code>parse_frugal_diff(output: &str;) -> Result, String>; apply_hunk_to_content(content, hunk).</code>
Antigravity Prompt:	<i>"Implement strict <<<< SEARCH / ===== / >>>>> REPLACE diff parser in Rust. Support indentation-insensitive fallback and multi-hunk safety."</i>
Test Command:	<code>cd src-tauri && cargo test diff::frugal_parser</code>

DAY 9: MULTI-FILE DIFF REVIEW UI

Objective:	Build interactive diff review interface in Monaco supporting inline annotations and side-by-side reviews.
Deliverables:	DiffReviewModal.tsx, Monaco deltaDecorations for hunks, Accept/Reject keyboard actions (Alt+N, Alt+P, Ctrl+Enter).
Interface / Contract:	<code>interface FileDiffState { filePath: string; hunks: Array<{ id: string; status: 'pending' 'accepted' 'rejected' }> }</code>
Antigravity Prompt:	<i>"Create Monaco diff review UI. Add glyph margin accept/reject buttons, file list with +/- line counters, and disk sync."</i>
Test Command:	<code>npm run test -- -t 'DiffReview'</code>

DAY 10: SHADOW GIT CHECKPOINT SYSTEM

Objective:	Create invisible Git snapshot engine recording workspace state into refs/ai-checkpoints/ before any AI edit.
Deliverables:	src-tauri/src/git/checkpoint.rs (git2 backend), automatic GC (7-day pruning, 100 max per branch).
Interface / Contract:	<code>create_checkpoint(workspace, summary) -> Result; list_checkpoints(workspace) -> Vec.</code>
Antigravity Prompt:	<i>"Implement Shadow Git checkpoints in Rust using git2. Commit to refs/ai-checkpoints/ without touching user HEAD. Add 7-day auto-GC."</i>
Test Command:	<code>cd src-tauri && cargo test git::checkpoint</code>

DAY 11: 1-CLICK TIME TRAVEL & ROLLBACK UX

Objective:	Build visual checkpoint timeline panel with hover diff previews and instant one-click restoration.
Deliverables:	CheckpointTimeline.tsx, diff popover preview, restore snapshot action with dirty-buffer stash protection.
Interface / Contract:	<code>restore_checkpoint(commit_hash) -> reverts working tree and reloads Monaco buffers safely.</code>
Antigravity Prompt:	<i>"Build Checkpoint Timeline in React. Render vertical snapshot history, diff previews, pin buttons, and 1-click restore with stash prompt."</i>
Test Command:	<code>npm run test -- -t 'CheckpointTimeline'</code>

DAY 12: TREE-SITTER AST SLICING & STRUCTURAL REPO MAP

Objective:	Build AST parser stripping function bodies and compressing 50+ files into an 800-token structural skeleton.
Deliverables:	src-tauri/src/ast/slicer.rs (Tree-sitter for TS, Py, Rust, Go), 85% input token reduction repo mapper.
Interface / Contract:	<code>generate_repo_skeleton(files: Vec) -> Result; strips method bodies, preserves types/signatures.</code>
Antigravity Prompt:	<i>"Implement Tree-sitter AST Slicing in Rust. Parse TS/Python/Rust, strip function bodies to '{ ... }', and assemble 800-token repo map."</i>
Test Command:	<code>cd src-tauri && cargo test ast::slicer</code>

DAY 13: HYBRID BM25 + VECTOR RETRIEVAL ENGINE (@CODEBASE)

Objective:	Build 2-stage retrieval pipeline combining nomic-embed-text embeddings, SQLite-vec, and BM25 cross-filtering (< 50ms).
Deliverables:	src-tauri/src/rag/ (embeddings.rs, sqlite_vec.rs, bm25.rs), query_codebase command returning top 3 snippets.
Interface / Contract:	query_codebase(query, max_results) -> Vec; Stage 1: vector top 15; Stage 2: BM25 top 3.
Antigravity Prompt:	<i>"Implement 2-stage RAG in Rust. Chunk files into 40-line windows, embed via nomic-embed-text, store in SQLite-vec, re-rank with BM25."</i>
Test Command:	cd src-tauri && cargo test rag::hybrid_retrieval

DAY 14: CHAT PANEL WITH @CODEBASE CONTEXT & IN-EDITOR INSERTION

Objective:	Implement primary AI Chat Assistant with @codebase context retrieval and 1-click diff insertion into Monaco.
Deliverables:	ChatPanel.tsx, @ mentions autocomplete, streaming markdown renderer, 'Apply to Editor' diff bridge.
Interface / Contract:	User types '@codebase how does auth work?' -> injects top 3 chunks + AST skeleton -> streams response.
Antigravity Prompt:	<i>"Build AI Chat Panel with @codebase mention triggers, streaming SSE display, and 1-click 'Apply to Editor' button routing to diff engine."</i>
Test Command:	npm run test -- -t 'ChatPanel'

WEEK 3: LANGUAGE INTELLIGENCE & DYNAMIC VRAM LIFECYCLE

DAY 15: EMBEDDED RUST LSP CLIENT CORE

Objective:	Implement native JSON-RPC 2.0 Language Server Protocol client in Rust to provide compiler-grade intelligence.
Deliverables:	src-tauri/src/lsp/client.rs (stdio pipe transport), auto-detection for rust-analyzer, pyright, tsserver.
Interface / Contract:	<code>LSP lifecycle: initialize, didOpen, didChange, didClose, shutdown; emits diagnostic events.</code>
Antigravity Prompt:	<i>"Implement native Rust LSP JSON-RPC client communicating over stdio. Auto-detect typescript-language-server, pyright, rust-analyzer."</i>
Test Command:	<code>cd src-tauri && cargo test lsp::client</code>

DAY 16: LSP DIAGNOSTICS, HOVER & GO-TO-DEFINITION

Objective:	Wire LSP diagnostic feeds, hover cards, and definition navigation into Monaco Editor.
Deliverables:	diagnosticsBridge.ts (Monaco markers), Monaco HoverProvider, Monaco DefinitionProvider.
Interface / Contract:	<code>Monaco editor receives real-time red/yellow squiggles; F12 jumps to cross-file symbol definitions.</code>
Antigravity Prompt:	<i>"Connect Rust LSP diagnostics to Monaco markers. Implement HoverProvider (type info) and DefinitionProvider (go-to-def)."</i>
Test Command:	<code>npm run test -- -t 'diagnosticsBridge'</code>

DAY 17: HARDWARE PROFILER & DYNAMIC VRAM SENTINEL

Objective:	Build real-time GPU/RAM hardware monitor preventing OOM crashes by dynamically budgeting context windows.
Deliverables:	src-tauri/src/hardware/profiler.rs (NVML / Apple sysctl / Windows DXGI), VRAM budget sentinel status bar widget.
Interface / Contract:	<code>get_hardware_profile() -> HardwareProfile; clamps num_ctx to 8192 with Q8_0 KV cache if VRAM < 4.5GB.</code>
Antigravity Prompt:	<i>"Build Hardware Profiler detecting GPU & free VRAM. Calculate safe num_ctx budgets. Create React status bar widget showing VRAM allocation."</i>
Test Command:	<code>cd src-tauri && cargo test hardware::profiler</code>

DAY 18: DYNAMIC TASK-BASED AUTO-MODEL ROUTER

Objective:	Dispatch IDE tasks to optimal parameter sizes while managing Ollama keep_alive VRAM eviction.
Deliverables:	src-tauri/src/router/model_router.rs: Typing->1.5B (pinned), Edit->7B (3m), Reasoning->8B (3m), Vision->2B (3m).
Interface / Contract:	<code>route_task(intent: TaskIntent, override: Option) -> ModelConfig with eviction priority manager.</code>
Antigravity Prompt:	<i>"Build Task-Based Model Router. Auto-route typing events to Qwen-1.5B (pinned) and edits to Qwen-7B. Evict heavy models after 3m on <=6GB GPUs."</i>
Test Command:	<code>cd src-tauri && cargo test router::model_router</code>

DAY 19: TERMINAL AUTO-FIX LOOP

Objective:	Capture test/build runtime errors from terminal output and trigger 1-click AI repair diffs.
Deliverables:	errorCapture.ts (regex parser for pytest, cargo, npm, tsc), 1-click 'Auto-Fix with AI' terminal action.
Interface / Contract:	<code>Detects failure at file.ts:42 -> prompts Qwen-7B with error context -> generates frugal diff.</code>
Antigravity Prompt:	<i>"Build terminal error capture for pytest, cargo, and npm. Extract file/line numbers and render 'Fix with AI' button triggering diff repair."</i>
Test Command:	<code>npm run test -- -t 'errorCapture'</code>

DAY 20: LOCAL WHISPER VOICE DICTATION

Objective:	Integrate 100% offline push-to-talk voice prompt dictation using embedded whisper.cpp (< 75MB RAM).
Deliverables:	src-tauri/src/audio/ (cpal audio capture, whisper.cpp bindings), global Ctrl+Space push-to-talk hook.
Interface / Contract:	<code>Ctrl+Space record -> transcribe local audio buffer -> stream text into editor or chat panel.</code>
Antigravity Prompt:	<i>"Integrate whisper.cpp and cpal in Rust. Capture mic audio on Ctrl+Space hold, run local offline transcription, and insert into editor/chat."</i>
Test Command:	<code>cd src-tauri && cargo test audio::whisper</code>

DAY 21: LIVE GENERATIVE SANDBOXED CANVAS

Objective:	Build isolated live-preview canvas rendering AI-generated React, Tailwind HTML, and Mermaid charts.
Deliverables:	LiveCanvas.tsx (sandboxed iframe, CSP no-network), local Babel compiler, split-view canvas panel.
Interface / Contract:	<code>Safe sandboxed iframe rendering generated UI components without remote network access.</code>
Antigravity Prompt:	<i>"Build Live Canvas in React using an isolated iframe with strict CSP. Bundle in-memory Babel and Tailwind to render UI snippets live."</i>
Test Command:	<code>npm run test -- -t 'LiveCanvas'</code>

WEEK 4: AGENT PROTOCOLS, ENTERPRISE AIR-GAP & DISTRIBUTION

DAY 22: MCP (MODEL CONTEXT PROTOCOL) CLIENT & GUARDRAILS

Objective:	Implement native Anthropic MCP client with strict JSON schema validation and retry prompting for local 7B models.
Deliverables:	src-tauri/src/mcp/client.rs (stdio transport), schema validator, one-tool-at-a-time fallback retries.
Interface / Contract:	<code>execute_mcp_tool(call: McpToolCall) -> Result; catches schema violations before execution.</code>
Antigravity Prompt:	<i>"Implement MCP Client in Rust over stdio for SQLite and Filesystem servers. Validate tool call JSON against schemas and add retry fallback."</i>
Test Command:	<code>cd src-tauri && cargo test mcp::client</code>

DAY 23: TYPESCRIPT PLUGIN ARCHITECTURE & EXTENSION HOST

Objective:	Build extensibility engine allowing community plugins to contribute UI panels, status bar items, and commands.
Deliverables:	PluginHost.ts, OpenStudioPlugin interface, sandboxed API (EditorAPI, TerminalAPI, CommandAPI), .osp bundle loader.
Interface / Contract:	<code>interface OpenStudioPlugin { id: string; activate(ctx: PluginContext): void; deactivate(): void; }</code>
Antigravity Prompt:	<i>"Implement PluginHost in TypeScript. Provide sandboxed APIs for editor, terminal, and UI panels. Support drag-and-drop .osp bundles."</i>
Test Command:	<code>npm run test -- -t 'PluginHost'</code>

DAY 24: ZERO-TELEMETRY AIR-GAP ENFORCEMENT & SECURITY CI

Objective:	Enforce air-gapped compliance: automated network audit script, local SQLite audit logger, and .openstudio/rules.yaml.
Deliverables:	scripts/verify-airgap.sh (fails CI on unannotated network calls), audit_logger.rs, rules.yaml policy engine.
Interface / Contract:	<code>CI fails if unannotated network call found; rules.yaml excludes .env and secrets from AI context.</code>
Antigravity Prompt:	<i>"Build air-gap verification script scanning for network calls. Implement local encrypted SQLite audit log. Enforce rules.yaml context boundaries."</i>
Test Command:	<code>bash scripts/verify-airgap.sh && cd src-tauri && cargo test security::audit_logger</code>

DAY 25: PERFORMANCE BENCHMARKING HARNESS

Objective:	Build automated performance test suite validating all latency, indexing, and memory claims.
Deliverables:	bench/ttft.py (<40ms), bench/diff_latency.py (<2s), bench/index_build.py (<60s for 10k files), BENCHMARKS.md generator.
Interface / Contract:	<code>Automated scripts benchmark P50/P95 latency and memory footprint; asserts all meet v2.0 specs.</code>
Antigravity Prompt:	<i>"Create benchmark suite in bench/. Test autocomplete TTFT, 7B diff generation, and repo indexing speed. Generate BENCHMARKS.md."</i>
Test Command:	<code>python bench/ttft.py && python bench/diff_latency.py</code>

DAY 26: STANDALONE VS CODE EXTENSION WEDGE

Objective:	Package Model Router, Token Optimizer, and Shadow Git as a VS Code extension for immediate developer distribution.
Deliverables:	extensions/vscode/ (package.json, extension.ts), inline completion provider, frugal diff commands, .vsix bundle.
Interface / Contract:	Drop-in VS Code extension providing local Qwen-1.5B autocomplete and shadow checkpoints inside standard VS Code.
Antigravity Prompt:	"Build standalone VS Code extension in extensions/vscode/. Reuse frugal diff parser and FIM formatter to offer local AI in VS Code."
Test Command:	cd extensions/vscode && npm run compile && npm run test

DAY 27: CROSS-PLATFORM PACKAGING & INSTALLER

Objective:	Configure Tauri v2 release bundles for Windows (.msi/.exe), macOS (.dmg Metal), and Linux with code signing setup.
Deliverables:	tauri.conf.json bundle configuration, .github/workflows/release.yml automated multi-platform release builder.
Interface / Contract:	Produces standalone installers for Windows x64, macOS arm64/x64, and Linux deb/AppImage.
Antigravity Prompt:	"Configure Tauri v2 multi-platform packaging in tauri.conf.json. Set up GitHub Actions CI workflow to build release installers on git tag."
Test Command:	npm run tauri build

DAY 28: PRODUCTION HARDENING, DOCS & LAUNCH KIT

Objective:	Complete end-to-end audit, finalize documentation, and prepare community launch campaign.
Deliverables:	README.md with demo recordings, SECURITY.md, PRIVACY.md, CONTRIBUTING.md, r/LocalLLaMA & HN Show HN launch post.
Interface / Contract:	Full test suites pass across Rust & TypeScript; zero compilation warnings; production tag v1.0.0-GA.
Antigravity Prompt:	"Perform final hardening pass. Verify all unit tests and benchmarks pass. Generate production documentation and Show HN / LocalLLaMA launch kit."
Test Command:	npm run test && cd src-tauri && cargo test && bash scripts/verify-airgap.sh

ANTIGRAVITY ONE-SHOT EXECUTION PROTOCOL

To execute this blueprint with Antigravity: **1) Target One Day at a Time:** Prompt Antigravity with 'Execute Day X of OPEN_STUDIO_EXECUTION_MASTERPLAN.md'. **2) Strict Deliverable Enforcement:** Ensure all files in Deliverables and Interface contracts are generated without placeholders. **3) Automated Test Gate:** Run the exact Test Command listed for the day. Do not proceed to Day X+1 until the test command exits with code 0.